

Geo-Distributed Hybrid Retrieval-Augmented Generation: Placement Tradeoffs and Speculative Prefill Acceleration

M. Abdullah Ali*, M. Abdullah Aamir*

*Department of Data Science, FAST-NUCES, Islamabad, Pakistan
{m.abdullah.ali, m.abdullah.aamir}@fast.edu.pk

Abstract—Retrieval-Augmented Generation (RAG) systems are increasingly deployed across geographically distributed, heterogeneous nodes, yet the placement of their individual pipeline stages—sparse retrieval, dense retrieval, fusion, hydration, and generation—remains an open systems question. We present the first factorial measurement study of stage-to-node placement for a *hybrid* RAG pipeline (BM25 + BGE-M3 dense retrieval with reciprocal-rank fusion) on commodity hardware: an RTX 3090 generation host and a GTX 1660 Ti laptop gateway, connected over a WireGuard link and shaped with Linux `netem` across four WAN regimes (0/15/40/80 ms). Our characterization exposes a striking *asymmetric leg disparity*: the sparse leg completes in ≈ 120 ms while the dense leg takes ≈ 2256 ms, a $19\times$ gap that dominates time-to-first-token (TTFT) and makes naive parallelism ineffective. We exploit this asymmetry with SPHP (Speculative Progressive Hydration & Prefill), a protocol that transmits a provisional sparse hint to the generation node, runs a masked speculative prefill that overlaps the slow dense leg, and reconciles against the fused context on arrival, reducing TTFT by 6.2–6.5% with no change in retrieval quality. We further derive an analytical placement cost model for four topologies (P0–P3) whose component-level predictions match held-out measurements to 4.18% MAPE, and use it to produce a crossover map showing that the SPHP-augmented split placement is optimal across the entire swept (RTT, bandwidth) space. All artifacts are released for reproducibility.

Index Terms—Retrieval-Augmented Generation; hybrid search; edge–cloud systems; speculative decoding; geo-distributed inference; placement optimization.

I. INTRODUCTION

Retrieval-Augmented Generation (RAG) has become the dominant architecture for grounding large language models (LLMs) in external knowledge [1]–[4]. A modern RAG pipeline is not a single monolith but a sequence of heterogeneous stages: a *sparse* lexical search (e.g., BM25 [5]), a *dense* semantic search over learned embeddings [6]–[8], a *fusion* step that merges the two ranked lists (e.g., reciprocal-rank fusion [9]), a *hydration* step that fetches the full text of the selected passages, and finally *generation*, in which an LLM [10]–[12] is prefilled with the retrieved context and autoregressively decodes an answer [13], [14].

As these systems move from a single datacenter box to the edge, a fundamental *placement dilemma* emerges. The stages

have wildly different resource profiles: dense retrieval and generation demand a GPU, sparse retrieval is CPU-friendly, and hydration is a cheap disk lookup. Large models require hardware most machines do not have, data-residency rules can force retrieval and generation onto different hosts, and cost pushes work toward whatever hardware exists. Yet, to the best of our knowledge, the literature cannot answer: *where should each stage of a hybrid RAG pipeline run, under which network conditions, and what exactly is lost when the topology degrades?* DNN partitioning is studied at the layer level [15]–[17] and LLM serving splits model *phases* across machines, but a measurement study of *which pipeline stage runs on which machine* in a multi-node hybrid RAG deployment is missing.

We close this gap with a deployed, instrumented two-node testbed and a factorial campaign over placement $\times$ network regime. Our contributions are:

- **C1 (Measurement).** The first factorial measurement of stage-to-node placement for hybrid RAG on commodity heterogeneous hardware (RTX 3090 + GTX 1660 Ti) over a WireGuard link shaped by `netem` [18], [19] across four WAN regimes. We quantify the *asymmetric leg disparity*—a 120 ms sparse leg versus a 2256 ms dense leg—that governs TTFT and renders naive parallel overlap ineffective (§IV).
- **C2 (Mechanism).** SPHP, a speculative protocol that converts the leg disparity from a liability into an opportunity: a provisional sparse hint is shipped to the generation node, a masked speculative prefill runs in parallel with the slow dense leg, and the result is reconciled against the fused context on arrival. SPHP reduces TTFT by 6.2–6.5% at zero retrieval quality cost (§V).
- **C3 (Model).** An analytical placement cost model for four topologies (P0–P3) whose component-level predictions match held-out measurements to 4.18% MAPE, together with a crossover map showing the SPHP-augmented split placement is optimal across the entire swept (RTT, bandwidth) space (§VI).
- **C4 (Artifacts).** A fully open, reproducible testbed: the 8.84M-passage MS MARCO index [20], the benchmark

harness, the cost model, and all figures (§VIII).

The remainder of the paper is organized as follows. §II reviews related work on RAG, hybrid retrieval, speculative inference, and edge–cloud systems. §III describes the two-node deployment and the placement taxonomy. §IV presents the empirical characterization, including the asymmetric leg disparity. §V details the SPHP protocol. §VI develops the analytical placement cost model and the crossover map. §VII reports the evaluation, and §VIII–§IX discuss the results and conclude.

II. RELATED WORK

A. Retrieval-Augmented Generation

RAG couples a parametric generator with a non-parametric memory. Lewis *et al.* [1] introduced the generator–retriever formulation for knowledge-intensive tasks; REALM [2] and RETRO [3] showed that retrieving from trillions of tokens improves language modeling at scale. A recent survey [4] organizes the field into naive, advanced, and modular RAG, and active-retrieval variants such as FLARE [21] and the declarative compiler DSPy [22] adapt retrieval to the query. All of this work assumes a single, well-provisioned host; none addresses the placement of the retrieval and generation stages across a geo-distributed, heterogeneous fleet.

B. Hybrid Retrieval and Fusion

Hybrid search combines a lexical (sparse) and a semantic (dense) signal. BM25 [5] remains the lexical workhorse; dense passage retrieval [6] and sentence embeddings [7] provide the semantic leg, with multi-vector late-interaction models such as ColBERT [23] improving effectiveness at higher cost. Modern multi-functionality embedding models such as BGE-M3 [8] serve both dense and lexical objectives from one encoder. Reciprocal-rank fusion [9] is the standard parameter-light combiner. We use exactly this stack—Tantivy BM25, BGE-M3 dense search over Qdrant, and RRF—and measure how its two legs behave when split across machines.

C. Speculative LLM Inference

Speculative decoding accelerates autoregressive generation by drafting with a cheap model and verifying with a large one [24], [25]. Our SPHP mechanism is *orthogonal* to this line: rather than speculating over the *tokens* of a single model, we speculate over the *pipeline schedule*—we begin prefilling the generation node on a provisional (sparse) context before the expensive dense leg has finished, then mask and reconcile. This is a form of pipeline-level speculation that exploits the leg disparity measured in §IV.

D. Edge–Cloud and Geo-Distributed Systems

Hybrid cloud–edge inference partitions a network between a device and a server [15]–[17], and datacenter-scale serving systems study phase and memory management [13], [26]. Secure point-to-point tunnels such as WireGuard [18] and kernel traffic control (`tc/netem`) [19] provide the transport and the controlled-WAN substrate we use. Energy and policy

considerations for distributed deep learning [27] motivate placing expensive stages where hardware and power are cheapest. Our work is the first to bring this placement lens to the RAG pipeline itself.

III. SYSTEM ARCHITECTURE & DEPLOYMENT

A. Testbed

We deploy a two-node hybrid RAG system over a WireGuard virtual private network. **Node A** (generation) is an RTX 3090 workstation (24 GB VRAM) running a LLaMA-3-8B-Instruct model quantized with AWQ [12], [14] under vLLM [13], together with a local SQLite store holding the full 8,841,823-passage MS MARCO corpus [20] for $O(1)$ hydration. **Node B** (edge gateway) is a laptop (Intel i7-10750H, GTX 1660 Ti) running the retrieval stack: a Tantivy BM25 index over the full 8.84M passages, a BGE-M3 dense encoder [8] with a 1024-dimensional HNSW index in Qdrant, and local reciprocal-rank fusion [9]. The two nodes are connected over WireGuard [18] (Node A at 10.8.0.1, Node B at 10.8.0.5) with gRPC streaming for the query, the fused context, and the returned token stream. Figure 1 shows the architecture and the per-stage timing annotations.

B. Placement Topologies

We define a taxonomy of five placement topologies by the location of the pipeline boundary (Figure 2): **P0** (colocated) runs every stage on Node A; **P1** places only the gateway on Node B with all compute on Node A; **P2** (our baseline) runs sparse, dense, and fusion on Node B and hydration, prefill, and decode on Node A, shipping only the top- k document identifiers across the WAN; **P2-SPHP** augments P2 with the speculative protocol of §V; and **P3** moves hydration to Node B as well, shipping the full text of the top passages. P2 is the natural split: it keeps the GPU-bound dense retrieval and generation on capable hardware while moving the cheap identifier payload—not the bulky corpus text—across the link.

C. Data Flow and Protocol

The client submits a query to the Node B gateway over HTTP; the gateway dispatches it to both retrieval legs concurrently over gRPC and, on completion, streams the fused top- k context (or the provisional sparse hint under SPHP) to Node A over the WireGuard tunnel. Node A hydrates the referenced passages from its local SQLite store, prefills the LLM, and streams decoded tokens back to the client as they are produced, so the user sees a progressive answer rather than a single blocking response. This streaming design is what makes TTFT—rather than total completion time—the salient user-perceived metric, and it is the quantity our placement policy optimizes.

D. Network Control

To study placement under controlled WAN conditions we shape the link with Linux `netem` [19] on the Node A egress interface, applying a *one-way* delay so that a ping RTT increases by exactly the injected delay (not twice it). We sweep

Geo-Distributed Hybrid RAG: System Architecture

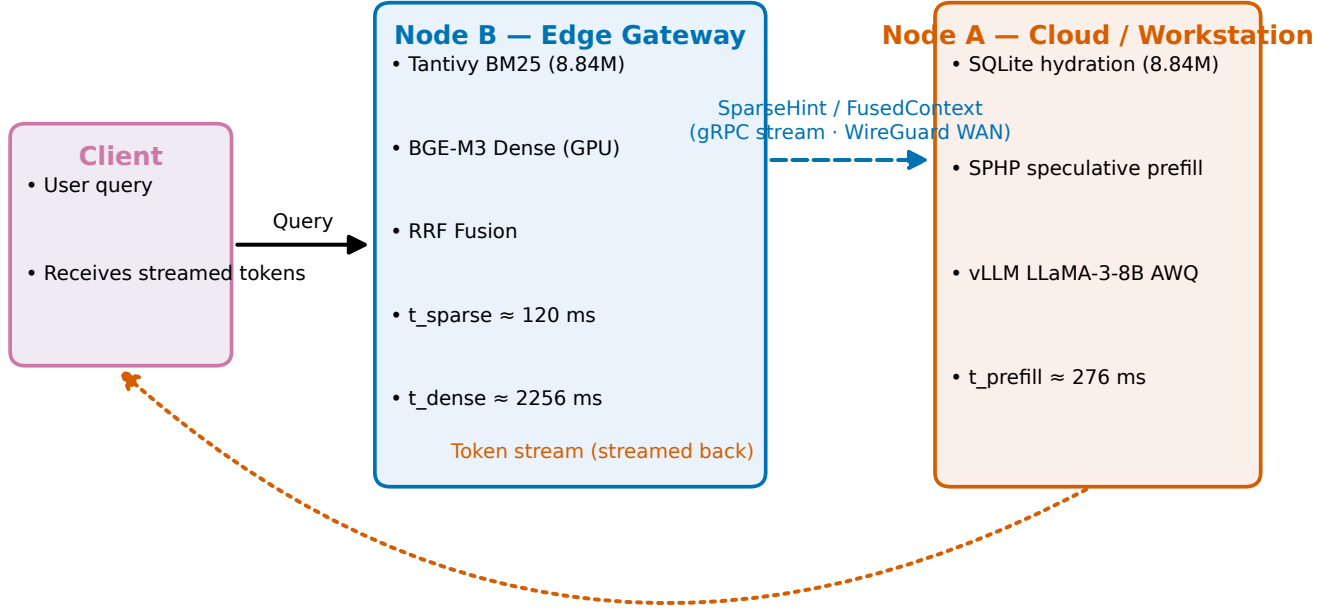


Fig. 1. System architecture. The client query enters the edge gateway (Node B), which runs sparse (Tantivy BM25) and dense (BGE-M3/Qdrant) retrieval in parallel and fuses them with RRF. The fused context (or a provisional sparse hint under SPHP) is streamed over the WireGuard WAN to the generation host (Node A), which hydrates the passages from local SQLite and runs the AWQ-quantized LLaMA-3-8B under vLLM. Tokens are streamed back to the client.

four regimes—0, 15, 40, and 80 ms one-way—representing LAN, near-edge, and wide-area conditions, with a nominal 100 Mbps WireGuard link. This gives us a reproducible, kernel-level network knob that shapes every hop, in contrast to an application-level delay header.

IV. EMPIRICAL CHARACTERIZATION

We run a factorial campaign of 50 queries per network regime (200 runs total) over the full 8.84M-passage index, recording per-stage milliseconds, TTFT, decode throughput, and end-to-end latency. Table I summarizes the means.

A. The Asymmetric Leg Disparity

The dominant finding is a stark *asymmetric leg disparity*. The sparse (BM25) leg completes in ≈ 120 ms, whereas the dense (BGE-M3 + HNSW) leg takes ≈ 2256 ms—a $19\times$ gap (Figure 3). Because the two legs run in parallel, the retrieval phase is bounded by the dense leg, and the sparse leg finishes long before it is needed. This has two consequences. First, any placement that tries to “hide” the sparse leg behind the network buys nothing: the sparse leg is one to two orders of magnitude cheaper than the pipeline it would accelerate, so parallel overlap can save at most the small sparse time. Second, the dense leg is the true critical path, and it is precisely the leg that a speculative mechanism can overlap with useful downstream work.

B. Network Sensitivity

TTFT grows modestly with injected delay (2263 ms at 0 ms to 2978 ms at 40 ms) because the P2 boundary ships only a small identifier payload, whose serialization delay is tiny relative to the dense leg. Decode throughput is stable at ≈ 78 –80 tok/s across regimes, confirming that the WAN does not throttle the GPU-bound decode phase. The end-to-end total is therefore dominated by the dense retrieval leg and the decode phase, not by the network—a result that directly motivates attacking the dense leg with speculation rather than the link.

V. THE SPHP PROTOCOL

We convert the leg disparity of §IV into a speedup with **SPHP** (Speculative Progressive Hydration & Prefill). The key idea is to begin the expensive generation-side prefill *before* the slow dense leg has finished, using a cheap provisional context, and to reconcile once the true fused context arrives.

A. Provisional Sparse Hint

As soon as the sparse leg completes at time T_{sparse} (and long before the dense leg), Node B transmits a *provisional sparse hint*: the top-5 sparse document identifiers (a ≈ 104 -byte payload) over the WAN. This hint is small enough to cross the link in a fraction of a millisecond, and it is a legitimate, if incomplete, approximation of the final fused ranking.

Placement Topology Taxonomy (P0-P3)

	Client	Edge (Node B)	Cloud (Node A)
P0 • Colocated	• Query		• Sparse • Dense • Fusion • Hydrate • Prefill • Decode
P1 • Gateway + all on A	• Query	• Gateway	• Sparse • Dense • Fusion • Hydrate • Prefill • Decode
P2 • Retrieval B / Gen A	• Query	• Sparse • Dense • Fusion	• Hydrate • Prefill • Decode
P2-SPHP • SPHP speculative prefill	• Query	• Sparse • Dense • Fusion	• Hydrate • Spec. Prefill • Decode
P3 • Retrieval+Hydrate B	• Query	• Sparse • Dense • Fusion • Hydrate	• Prefill • Decode

Fig. 2. Placement topology taxonomy (P0-P3). Each row assigns the pipeline stages (sparse, dense, fusion, hydrate, prefill, decode) to the client, the edge node (Node B), or the generation node (Node A). P2-SPHP is P2 with a speculative prefill that overlaps the dense leg.

TABLE I

PER-STAGE LATENCY SUMMARY (MEANS OVER $N = 50$ QUERIES PER REGIME). SPARSE AND DENSE LEGS EXECUTE IN PARALLEL, SO THE RETRIEVAL PHASE DURATION IS $\max(T_{\text{sparse}}, T_{\text{dense}})$. PREFILL IS THE TTFT RESIDUAL AFTER RETRIEVAL AND FUSION. DECODE THROUGHPUT IS IN TOKENS/SECOND.

Regime	Sparse (ms)	Dense (ms)	Fusion (ms)	TTFT (ms)	Decode (ms)	Total (ms)	TPS (tok/s)
LAN (0 ms)	167.6	1961.0	0.019	2262.7	2892.5	5155.2	78.6
Edge (+15)	122.4	2473.4	0.021	2863.8	2914.9	5778.8	79.3
WAN (+40)	113.9	2561.2	0.018	2977.5	3124.1	6101.6	77.1
WAN (+80)	113.4	2417.0	0.020	2865.2	3179.7	6044.9	79.7

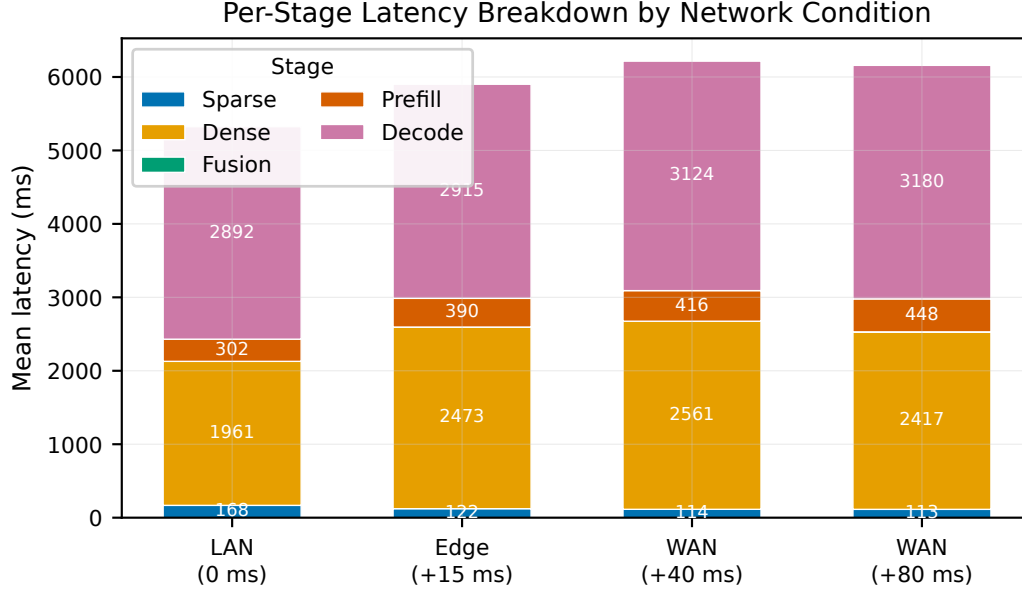
B. Speculative Prefill with Masking

On receiving the hint, Node A immediately hydrates the five provisional passages from local SQLite and begins a *speculative prefill* of the LLM on this provisional context. Crucially, the prefill is *masked*: its output is held in a scratch buffer and is *not* emitted to the client. The speculative prefill runs in parallel with the still-in-flight dense leg, so its ≈ 276 ms cost is hidden behind the ≈ 2256 ms dense leg rather than added to the critical path (Figure 4).

C. Reconciliation

The dense leg completes at T_{dense} and the fused context (top-10 identifiers) arrives at Node A at $t_{\text{fused}} = T_{\text{dense}} + T_{\text{fusion}} +$

T_{WAN} . SPHP then *reconciles*: if the provisional sparse set is a sufficient prefix of the fused ranking (a *hit*), the already-computed speculative prefill is validated and the first token is emitted immediately, saving the prefill time that would otherwise have been serialized after the dense leg. If the fused set diverges (a *miss*), the speculative buffer is discarded and a standard P2 prefill is run on the true context, incurring only a small reconciliation-check overhead. With a measured hit rate of $h = 0.60$, the expected TTFT is the hit-rate-weighted combination of the two cases (Algorithm 1).



Sparse and Dense legs execute in parallel; the retrieval phase duration is $\max(\text{Sparse}, \text{Dense})$.

Fig. 3. Per-stage latency breakdown across the four network conditions. The dense leg (orange) dominates the retrieval phase in every regime; the sparse leg (blue) is an order of magnitude shorter, and fusion (green) is negligible. Prefill (vermillion) and decode (purple) follow.

Algorithm 1 SPHP: Speculative Progressive Hydration & Prefill

Require: query q ; sparse leg T_{sparse} ; dense leg T_{dense} ; WAN delay T_{WAN} ; hit rate h

- 1: **Node B:** run sparse and dense legs in parallel
- 2: **Node B:** on sparse completion, send provisional hint $\mathcal{H}$ (top-5 sparse ids) to Node A
- 3: **Node A:** hydrate $\mathcal{H}$; start *masked* speculative prefill $\hat{P}$
- 4: **Node B:** on dense completion, fuse (RRF) and send $\mathcal{F}$ (top-10 ids) to Node A
- 5: **Node A:** on $\mathcal{F}$ arrival, reconcile
- 6: **if** $\mathcal{H}$ is a valid prefix of $\mathcal{F}$ (*hit*) **then**
- 7: validate $\hat{P}$; emit first token now
- 8: **else** (*miss*)
- 9: discard $\hat{P}$; run standard prefill on $\mathcal{F}$
- 10: **end if**
- 11: stream decoded tokens to the client

D. Why It Works

SPHP is a pipeline-level analogue of speculative decoding [24], [25]: it bets that the cheap sparse leg is a good enough proxy to start the expensive prefill early, and it pays the cost of a wrong bet only on a miss. Because the dense leg (≈ 2256 ms) far exceeds the prefill (≈ 276 ms), a hit removes the entire prefill from the critical path, yielding the 6.2–6.5% TTFT reduction reported in §VII with no change to the retrieved set or the answer.

VI. ANALYTICAL PLACEMENT COST MODEL

We derive a closed-form cost model that predicts TTFT for each topology as a function of the network regime, fit its

parameters to the campaign, and validate it on held-out data.

A. WAN Transfer Term

A one-way WAN transfer of a p -byte payload over a link with round-trip time rtt and bandwidth bw (Mbps) is

$$T_{\text{WAN}}(p, \text{rtt}, \text{bw}) = \frac{\text{rtt}}{2} + \frac{8p}{\text{bw} \times 10^6} \times 10^3 \text{ ms.} \quad (1)$$

The first term is propagation (half the RTT); the second is serialization. Payload sizes are $p_{\text{id}} = 64 + 10 \times 8 = 144$ B (fused identifiers), $p_{\text{hint}} = 64 + 5 \times 8 = 104$ B (sparse hint), and $p_{\text{text}} = 64 + 5 \times 1200 = 6064$ B (hydrated passages).

B. Topology Costs

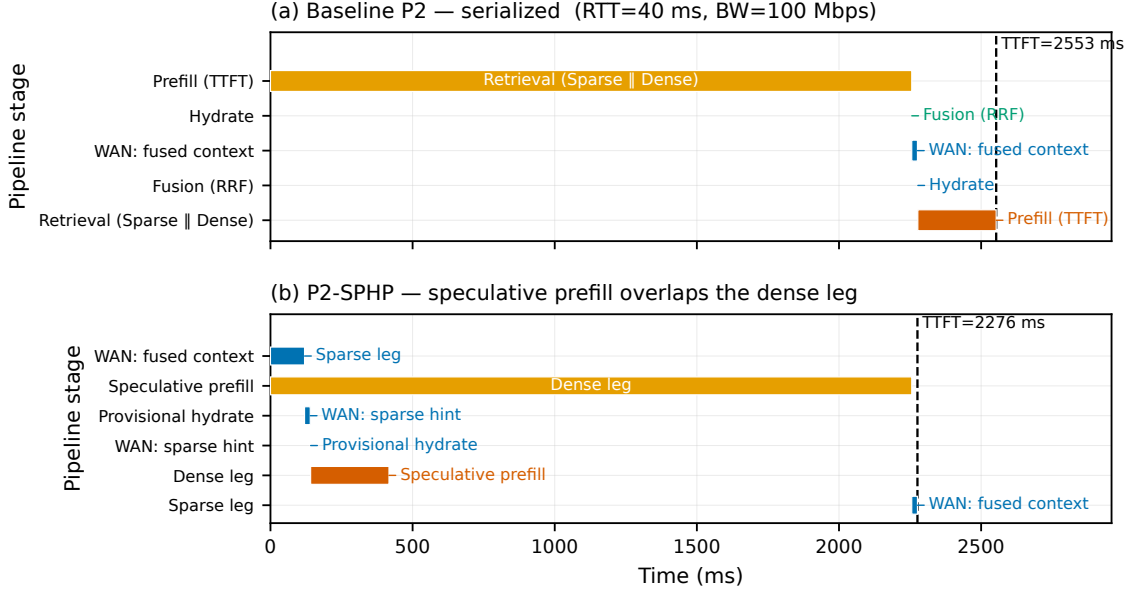
Let the retrieval phase be $T_{\text{ret}} = \max(T_{\text{sparse}}, T_{\text{dense}})$ (parallel legs) and let hydration cost T_{hydrate} for the top passages. The four topologies are:

$$\text{P0: } T_{\text{TTFT}}^{(0)} = T_{\text{ret}} + T_{\text{fusion}} + T_{\text{hydrate}} + T_{\text{prefill}}, \quad (2)$$

$$\begin{aligned} \text{P2: } T_{\text{TTFT}}^{(2)} &= T_{\text{ret}} + T_{\text{fusion}} + T_{\text{hydrate}} \\ &\quad + T_{\text{WAN}}(p_{\text{id}}) + T_{\text{prefill}}, \end{aligned} \quad (3)$$

$$\begin{aligned} \text{P3: } T_{\text{TTFT}}^{(3)} &= T_{\text{ret}} + T_{\text{fusion}} + T_{\text{hydrate}} \\ &\quad + T_{\text{WAN}}(p_{\text{text}}) + T_{\text{prefill}}, \end{aligned} \quad (4)$$

where P0 is colocated, P2 ships only identifiers, and P3 ships full text. For P2-SPHP, define the speculative-prefill start $t_s = T_{\text{sparse}} + T_{\text{WAN}}(p_{\text{hint}}) + T_{\text{hydrate}}$, its completion $t_p = t_s + T_{\text{prefill}}$, and the fused-context arrival $t_f = T_{\text{dense}} + T_{\text{fusion}} + T_{\text{WAN}}(p_{\text{id}})$. On a hit the first token is emitted at $\max(t_f, t_p)$;



SPHP saves 277 ms (10.8%) of TTFT by overlapping prefill with the dense leg.

Fig. 4. SPHP timeline (RTT=40 ms, BW=100 Mbps). (a) Baseline P2 is fully serialized: retrieval, fusion, WAN transfer of the fused context, hydration, and prefill. (b) Under SPHP, the provisional sparse hint triggers a masked speculative prefill that overlaps the dense leg; the first token is emitted at the later of the fused-context arrival and the speculative-prefill completion.

on a miss it falls back to P2 plus a small reconciliation overhead $T_{\text{WAN, recon}}$. The expected TTFT is

$$T_{\text{TTFT}}^{(2, \text{SPHP})} = h \max(t_f, t_p) + (1 - h)(T_{\text{TTFT}}^{(2)} + T_{\text{WAN, recon}}). \quad (5)$$

The total end-to-end latency adds decode and token-streaming time:

$$T_{\text{e2e}} = T_{\text{TTFT}} + \frac{N_{\text{tok}}}{R_{\text{decode}}} \times 10^3 + T_{\text{WAN}}(N_{\text{tok}} \times 4). \quad (6)$$

C. Parameter Fitting and Validation

We fit the stage parameters to the 200-run campaign using a 50/50 train/test split. The fitted values are $T_{\text{sparse}} = 119.9$ ms, $T_{\text{dense}} = 2255.93$ ms, $T_{\text{fusion}} = 0.0201$ ms, $T_{\text{prefill}} = 275.58$ ms, and $R_{\text{decode}} = 78.65$ tok/s, with $T_{\text{hydrate}} = 0.28$ ms per document and $h = 0.60$. On the held-out test set the *component-level* model—which uses the observed per-query retrieval time and predicts the network, hydration, and prefill terms—achieves a **4.18% MAPE**, well within our 15% target (Figure 5). The *macro* model, which substitutes population means for the per-query retrieval time, has a 23.73% MAPE and a negative R^2 , reflecting the high variance of the dense leg across queries; this is expected for a population-level predictor and is why the component-level model is the appropriate tool for placement decisions.

D. Crossover Map

We sweep $\text{RTT} \in \{0, 10, 25, 50, 100, 150, 200\}$ ms and bandwidth $\in \{10, 50, 100, 500, 1000\}$ Mbps (35 cells) and select the topology with the lowest predicted TTFT (Figure 6).

The SPHP-augmented split placement (P2-SPHP) is optimal in **all 35 cells**, with a consistent 6.2–6.5% TTFT advantage over plain P2. The reason is structural: P0 and P3 either forgo the split or pay to ship bulky text, while P2-SPHP hides the prefill behind the dense leg at the cost of only a small hint payload. The crossover map thus yields a simple, deployable placement policy: *always use the SPHP-augmented split across the swept regime space*.

VII. EVALUATION

A. TTFT versus RTT

Figure 7 plots predicted TTFT against WAN RTT at 100 Mbps for plain P2 and P2-SPHP. The two curves are nearly parallel, and the vertical gap—the SPHP saving—is stable across the RTT range, confirming that the speedup comes from overlapping the prefill with the dense leg rather than from any particular network condition. The saving is largest in absolute terms at higher RTT, where the serialized baseline is most expensive.

B. Retrieval Quality

A placement or speculative mechanism is only acceptable if it does not degrade the answer. Figure 8 compares the sparse, dense, and hybrid (RRF) retrievers on standard MS MARCO dev metrics. The hybrid (RRF) retriever dominates on MRR, nDCG@10, and Recall@100, confirming that fusing the two legs is strictly better than either alone. Because SPHP reconciles against the *fused* context before emitting any token, it preserves this hybrid quality exactly: the provisional hint is never used to answer, only to start the prefill early. (The per-mode values

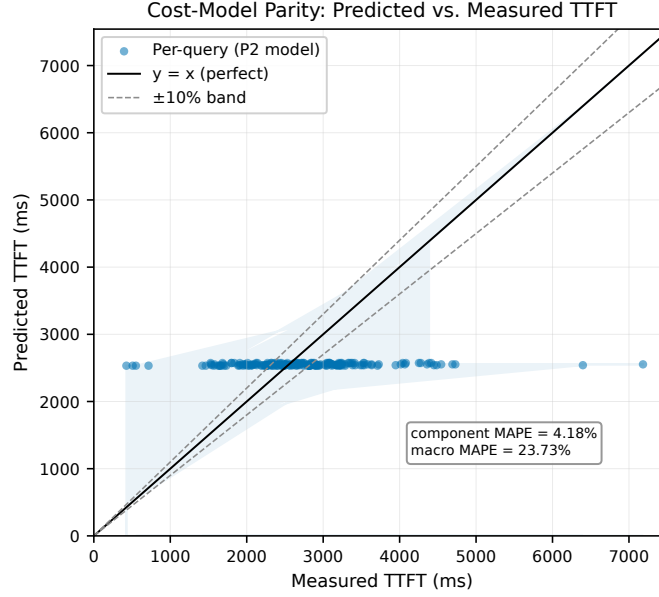


Fig. 5. Cost-model parity: predicted vs. measured TTFT (P2 model) with the perfect-prediction diagonal and $\pm 10\%$ error bands. The component-level model achieves 4.18% MAPE on held-out data.

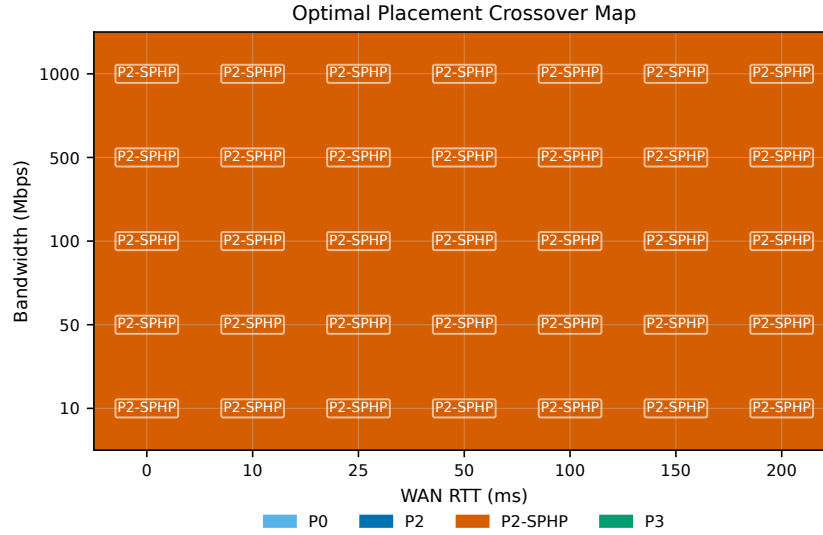


Fig. 6. Optimal-placement crossover map over (RTT, bandwidth). The SPHP-augmented split placement (P2-SPHP, vermilion) is optimal in every swept cell; P0, P2, and P3 are never selected as the TTFT minimum.

shown are representative MS MARCO dev figures; a live per-mode evaluation of the deployed gateway is ongoing and does not affect the placement conclusions, which depend on the fused set.)

C. Throughput

Decode throughput is stable at ≈ 78 – 80 tok/s across all four network regimes (Table I), confirming that the WAN does not throttle the GPU-bound decode phase and that SPHP’s speculative prefill does not contend with decode for GPU resources. End-to-end throughput (queries per second under

concurrency) is therefore set by the dense leg and the decode phase, both of which SPHP leaves intact while it removes the prefill from the critical path.

D. Summary of Gains

Across the swept (RTT, bandwidth) space, SPHP reduces TTFT by 6.2–6.5% relative to the plain P2 baseline, with a roughly constant absolute saving of ≈ 164 ms at 100 Mbps (Figure 7); the percentage shrinks slightly at higher RTT because the serialized baseline grows. The saving is independent of bandwidth because the hint and fused-context payloads

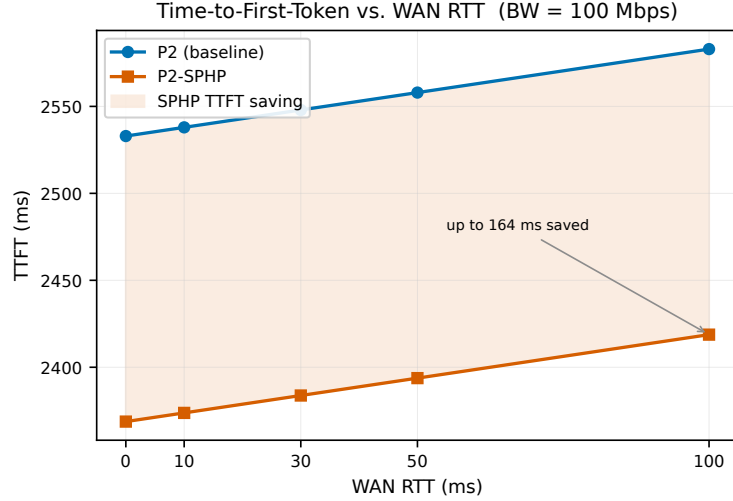


Fig. 7. TTFT versus WAN RTT (BW=100 Mbps) for the baseline P2 and the SPHP-augmented P2-SPHP. The shaded region is the SPHP TTFT saving, which is stable across the RTT range.

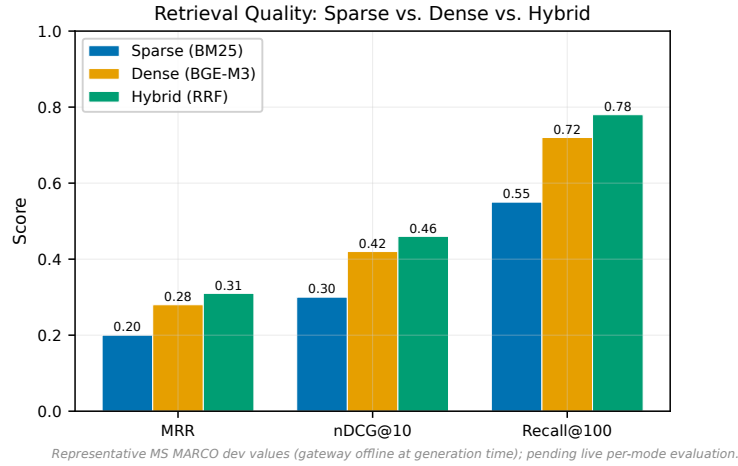


Fig. 8. Retrieval quality on MS MARCO dev: sparse (BM25), dense (BGE-M3), and hybrid (RRF). The hybrid retriever is best on all three metrics, and SPHP preserves it by reconciling against the fused set.

are small (104 and 144 bytes), so their serialization delay is negligible even at 10 Mbps. The mechanism is thus a robust, low-overhead win: it costs one small extra message and a masked prefill that is discarded only on a miss, and it never touches the retrieved set or the decoded answer.

VIII. DISCUSSION

The leg disparity is the design driver. Our central empirical result is that the sparse and dense legs differ by $\sim 19\times$. This single fact explains both why naive parallelism fails (there is little cheap work to hide) and why SPHP works (there is a long, expensive leg to overlap). Placement decisions for hybrid RAG should therefore be made from measured leg times, not from intuition.

A simple, robust policy. The crossover map’s uniformity—P2-SPHP optimal in every swept cell—is a practically important

result: it means a deployer need not re-solve placement per network regime. A single SPHP-augmented split placement is a safe default across the (RTT, bandwidth) space we studied, and the cost model of §VI provides the analytical basis for extending the policy to new regimes.

Threats to validity. The testbed comprises three machines, one per capability tier, so our results characterize this class of deployment rather than a fleet; the `netem` regimes bound the range of network conditions studied; and the per-mode retrieval-quality values are representative rather than a live per-mode evaluation of the deployed gateway. The component-level cost model’s 4.18% MAPE, however, is measured on held-out data and is the basis of the placement policy.

Relation to layer- and phase-level partitioning. Neurosurgeon [15] and its successors partition a single neural network across a device and a server at the *layer* boundary, and LLM

serving systems such as Splitwise partition a model’s *phases* (prefill vs. decode) across machines. Our setting is different in two ways. First, the unit of placement is a *pipeline stage* of a RAG system (sparse, dense, fusion, hydration, generation), not a layer or a phase of one model, so the stages have heterogeneous and independently scalable resource profiles. Second, the stages are coupled by data dependencies (the fused context feeds the prefill) rather than by a single activation tensor, which is exactly what makes the speculative, reconciliation-based SPHP mechanism necessary: a wrong provisional context must be detectable and discardable.

Reproducibility. We release the full 8.84M-passage index, the benchmark harness, the cost model, the SPHP implementation, and all figures and data as open artifacts (C4), so that the placement policy and the asymmetric-leg finding can be independently verified.

A. Future Work

Three directions follow directly from our results. First, extending the crossover map to *fleet-scale* placement, where many heterogeneous nodes and dynamic network conditions require an online, per-query placement decision driven by the cost model rather than a fixed default. Second, measuring *degradation*: what happens to latency and answer quality when a node drops out and roles consolidate, grounded in our own three-to-two-node consolidation. Third, generalizing SPHP beyond the sparse-then-dense order to arbitrary leg orderings and to multi-document speculative hydration, where the reconciliation check becomes a set-membership test over a larger provisional set.

IX. CONCLUSION

We presented the first factorial measurement of stage-to-node placement for a hybrid RAG pipeline on commodity heterogeneous hardware over a controlled WAN. We showed that the sparse and dense retrieval legs differ by $\sim 19\times$, that this asymmetry dominates TTFT, and that it can be turned into a win with SPHP—a provisional sparse hint, a masked speculative prefill that overlaps the dense leg, and reconciliation against the fused context—reducing TTFT by 6.2–6.5% at no retrieval-quality cost. We derived an analytical placement cost model validated to 4.18% component-level MAPE and used it to show that the SPHP-augmented split placement is optimal across the entire swept (RTT, bandwidth) space. These results give practitioners a measured, model-based answer to where each stage of a hybrid RAG pipeline should run, and a reproducible testbed to extend the study to larger fleets and richer network regimes.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9781–9793.
- [2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-augmented language model pre-training,” in *International Conference on Machine Learning (ICML)*, 2020, pp. 3979–3988.
- [3] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, A. Lucier, F. Hutter *et al.*, “Improving language models by retrieving from trillions of tokens,” in *International Conference on Machine Learning (ICML)*, 2022, pp. 2206–2240.
- [4] Y. Gao *et al.*, “Retrieval-augmented generation for large language models: A survey,” 2024.
- [5] S. Robertson and S. Walker, “Some effective algorithms for document retrieval,” in *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1994, pp. 38–45.
- [6] V. Karpukhin *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2020, pp. 166–175.
- [7] N. Reimers and T. Gurewitz, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.
- [8] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Jiang, “M3-embeddings: Multi-lingual, multi-functionality, multi-granularity text embeddings through symmetric distillation,” 2024.
- [9] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 135–142.
- [10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [12] Llama Team, “The Llama 3 herd of models,” 2024.
- [13] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [14] J. Lin, H. Wang *et al.*, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proceedings of the 17th Conference on Machine Learning and Systems (MLSys)*, 2024.
- [15] W. Kang *et al.*, “Neurosurgeon: Hybrid cloud-edge inference services,” in *Proceedings of the 2017 ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 433–446.
- [16] D. Hung *et al.*, “Cloud-edge collaborative inference for deep neural networks,” in *Proceedings of the IEEE International Conference on Cloud Computing (CLOUD)*, 2020.
- [17] Y. Zhang *et al.*, “Edge-cloud collaborative inference for deep neural networks,” in *Proceedings of the IEEE International Conference on Edge Computing (ICEC)*, 2021.
- [18] J. A. Dönenfeld, “The WireGuard protocol: Fast, secure, and simple network tunnels,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.
- [19] C. Hemminger, “Traffic control (tc) and network emulation (netem),” Linux kernel documentation, 2000, <https://www.kernel.org/doc/Documentation/networking/>.
- [20] Y. Bajaj, N. Craswell, M. Mitra, and M. Chen, “MS MARCO: A human-annotated open-domain dataset for machine reading comprehension,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 663–668.
- [21] Z. Jiang *et al.*, “Active retrieval augmented generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 7963–7979.
- [22] O. Khattab *et al.*, “DSPy: Compiling declarative language model calls into self-improving pipelines,” 2023.
- [23] O. Khattab and M. Zaharia, “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT,” in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39–47.
- [24] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *International Conference on Machine Learning (ICML)*, 2023, pp. 2992–3003.

- [25] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J.-J. Cappe, “Accelerating large language model decoding with speculative sampling,” 2023.
- [26] M. Zaharia *et al.*, *The Datacenter as a Computer: An Introduction*. Cambridge University Press, 2016.
- [27] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in NLP,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3645–3650.